

Приложение

```
1. #include<iostream>
2. #include<string>
3.
4. class MyExeption : std::exception {
5. protected:
6.     std::string str;
7. public:
8.     MyExeption(const std::string& str) {
9.         this->str = str;
10.    }
11.    virtual ~MyExeption() {}
12.    virtual void print_exeption() {
13.        std::cerr << str << std::endl;
14.    };
15.};
16.
17. class condition_exeption : MyExeption {
18. public:
19.     condition_exeption() : MyExeption("conditions do not allow to do this") {
20.    }
21.    virtual ~condition_exeption() { print_exeption(); }
22.};
23.
24. class size_exeption : MyExeption {
25. public:
26.     size_exeption() : MyExeption("wrong size or out of bounds") {}
27.    virtual ~size_exeption() { print_exeption(); }
28.};
29.
30. class nullptr_exeption : MyExeption {
31. public:
32.     nullptr_exeption() : MyExeption("pointer to 0") {}
33.    virtual ~nullptr_exeption() { print_exeption(); }
34.};
35.
36. class memory_exeption : MyExeption {
37. public:
38.     memory_exeption() : MyExeption("no memory allocated") {}
39.    virtual ~memory_exeption() { print_exeption(); }
40.};
41.
42. template<class K, class T, class D>
43. class node {
44. protected:
45.     K key;
46.     T value;
47.     int size;
48.     D product;
49.     node<K, T, D>* left;
50.     node<K, T, D>* right;
51.     node<K, T, D>* parent;
```

```

51. D data;
52. public:
53. node<K, T, D>() {
54.     key = K();
55.     value = T();
56.     left = right = parent = NULL;
57.     size = 1;
58.     product = T();
59.     data = D();
60. }
61.
62. node<K, T, D>(K key, T value, D data) {
63.     this->key = key, this->value = value;
64.     left = right = parent = NULL;
65.     size = 1;
66.     product = data;
67.     this->data = data;
68. }
69.
70. ~node<K, T, D>() {
71.     //std::cout << "exit ( " << key << " " << value << " )" << std::endl;
72. }
73. node<K, T, D>* setparent(node<K, T, D>* prnt) { return parent = prnt; }
74.
75. node<K, T, D>* setleft(node<K, T, D>* l) { return left = l; }
76.
77. node<K, T, D>* setright(node<K, T, D>* r) { return right = r; }
78.
79. node<K, T, D>* getparent() { return parent; }
80.
81. node<K, T, D>* getleft() { return left; }
82.
83. node<K, T, D>* getright() { return right; }
84.
85. K getkey() { return key; }
86.
87. T getvalue() { return value; }
88.
89. int getsize() { return size; }
90.
91. void setsize(int s) { size = s; }
92.
93. D getproduct() { return product; }
94.
95. void setproduct(D prod) { product = prod; }
96.
97. D getdata() { return data; }
98.
99. void setdata(D data) { this->data = data; }
100.
101.     void setkey(K newkey) { key = newkey; }
102.
103.     void setvalue(T newvalue) { value = newvalue; }

```

```

104.
105.     node<K, T, D>* succs() {
106.         if (right == NULL) {
107.             node<K, T, D>* tmp = this;
108.             while (true) {
109.                 if (tmp->parent == NULL)
110.                     return tmp->parent;
111.                 else if (tmp->parent->key > key) {
112.                     return tmp->parent;
113.                 }
114.                 else {
115.                     tmp = tmp->parent;
116.                 }
117.             }
118.         }
119.         else {
120.             node<K, T, D>* tmp = this->right;
121.             while (tmp->left != NULL) {
122.                 tmp = tmp->left;
123.             }
124.             return tmp;
125.         }
126.     }
127.
128.     friend std::ostream& operator<<(std::ostream& stream, node<K, T, D>
    & node) {
129.         stream << "(" << node.key << " " << node.value << " " << node.dat
    a << " )" << std::endl;
130.         return stream;
131.     }
132.
133.     friend std::ostream& operator<<(std::ostream& stream, const node<K,
    T, D>& node) {
134.         stream << "(" << node.key << " " << node.value << " " << node.dat
    a << " )" << std::endl;
135.         return stream;
136.     }
137. };
138.
139. template<class K, class T, class D>
140. class treapiterator {
141. protected:
142.     node<K, T, D>* cur;
143. public:
144.     treapiterator<K, T, D>() {
145.         cur = NULL;
146.     }
147.     treapiterator<K, T, D>(node<K, T, D>* ptr) {
148.         cur = ptr;
149.     }
150.     treapiterator<K, T, D>& operator++(int v) {
151.         cur = cur->succs();
152.         return *this;

```

```

153.     }
154.
155.     treapiterator<K, T, D>& operator++(int v) const {
156.         cur = cur->sucs();
157.         return treapiterator<K, T, D>(cur);
158.     }
159.     treapiterator<K, T, D>& operator++() {
160.         cur = cur->sucs();
161.         return treapiterator<K, T, D>(cur);
162.     }
163.     treapiterator<K, T, D>& operator++() const {
164.         cur = cur->sucs();
165.         return treapiterator<K, T, D>(cur);
166.     }
167.     treapiterator<K, T, D>& operator=(node<K, T, D>* ptr) {
168.         cur = ptr;
169.         return *this;
170.     }
171.     treapiterator<K, T, D>& operator=(const node<K, T, D>* ptr) const {
172.         cur = ptr;
173.         return *this;
174.     }
175.     treapiterator<K, T, D>& operator=(const treapiterator<K, T, D>& ite
r) {
176.         cur = iter.cur;
177.         return *this;
178.     }
179.     treapiterator<K, T, D>& operator=(const treapiterator<K, T, D>& ite
r) const {
180.         cur = iter.cur;
181.         return *this;
182.     }
183.
184.     bool operator==(const node<K, T, D>* ptr) {
185.         return cur == ptr;
186.     }
187.     bool operator==(const node<K, T, D>* ptr) const {
188.         return cur == ptr;
189.     }
190.     bool operator!=(const node<K, T, D>* ptr) {
191.         return cur != ptr;
192.     }
193.     bool operator!=(const node<K, T, D>* ptr) const {
194.         return cur != ptr;
195.     }
196.
197.     bool operator==(const treapiterator<K, T, D>& ptr) {
198.         return cur == ptr.cur;
199.     }
200.     bool operator==(const treapiterator<K, T, D>& ptr) const {
201.         return cur == ptr.cur;
202.     }
203.     bool operator!=(const treapiterator<K, T, D>& ptr) {

```

```

204.         return cur != ptr.cur;
205.     }
206.     bool operator!=(const treapiterator<K, T, D>& ptr) const {
207.         return cur != ptr.cur;
208.     }
209.
210.     node<K, T, D>* operator*() {
211.         return cur;
212.     }
213.     node<K, T, D>* operator*() const {
214.         return cur;
215.     }
216. };
217.
218. template<class K, class T, class D>
219. class treap {
220. protected:
221.     node<K, T, D>* root;
222.
223.     //2)
224.     node<K, T, D>* merge2(const node<K, T, D>* L, const node<K, T, D>*
    R) {
225.         if (L == NULL || R == NULL) return (L) ? new node<K, T, D>(*L) : n
        ew node<K, T, D>(*R);
226.
227.         if (L->getvalue() > R->getvalue()) {
228.             if (L->getkey() < R->getkey()) {
229.                 node<K, T, D>* tmp = merge2(L->getright(), R);
230.                 if (tmp != NULL)
231.                     tmp->setparent(new node<K, T, D>(* L));
232.                 L->setright(tmp);
233.             }
234.             else {
235.                 node<K, T, D>* tmp = merge2(L->getleft(), R);
236.                 if (tmp != NULL)
237.                     tmp->setparent(new node<K, T, D>(* L));
238.                 L->setleft(tmp);
239.             }
240.             return new node<K, T, D>(*L);
241.         }
242.         else {
243.             if (L->getkey() < R->getkey()) {
244.                 node<K, T, D>* tmp = merge2(L, R->getleft());
245.                 if (tmp != NULL)
246.                     tmp->setparent(new node<K, T, D>(* R));
247.                 R->setleft(tmp);
248.             }
249.             else {
250.                 node<K, T, D>* tmp = merge2(L, R->getright());
251.                 if (tmp != NULL)
252.                     tmp->setparent(new node<K, T, D>(* R));
253.                 R->setright(tmp);
254.             }

```

```

255.         return new node<K, T, D>(* R);
256.     }
257. }
258.
259. //1)
260. node<K, T, D>* merge(node<K, T, D>* L, node<K, T, D>* R) {
261.     if (L == NULL || R == NULL) return (L) ? L : R;
262.
263.     if (L->getvalue() > R->getvalue()) {
264.         if (L->getkey() < R->getkey()) {
265.             node<K, T, D>* tmp = merge(L->getright(), R);
266.             if (tmp != NULL)
267.                 tmp->setparent(L);
268.             L->setright(tmp);
269.         }
270.         else {
271.             node<K, T, D>* tmp = merge(L->getleft(), R);
272.             if (tmp != NULL)
273.                 tmp->setparent(L);
274.             L->setleft(tmp);
275.         }
276.         return L;
277.     }
278.     else {
279.         if (L->getkey() < R->getkey()) {
280.             node<K, T, D>* tmp = merge(L, R->getleft());
281.             if (tmp != NULL)
282.                 tmp->setparent(R);
283.             R->setleft(tmp);
284.         }
285.         else {
286.             node<K, T, D>* tmp = merge(L, R->getright());
287.             if (tmp != NULL)
288.                 tmp->setparent(R);
289.             R->setright(tmp);
290.         }
291.         return R;
292.     }
293. }
294.
295. //1)
296. std::pair<node<K, T, D>*, node<K, T, D>*> split(node<K, T, D>* cur,
    K x) {
297.     if (cur == NULL) return std::pair<node<K, T, D>*, node<K, T, D>*>(
        NULL, NULL);
298.     else if (x > cur->getkey()) {
299.         std::pair<node<K, T, D>*, node<K, T, D>*> tmp = split(cur-
            >getright(), x);
300.         cur->setright(tmp.first);
301.         if (tmp.first != NULL)
302.             tmp.first->setparent(cur);
303.         if (tmp.second != NULL)
304.             tmp.second->setparent(NULL);

```

```

305.         return std::pair<node<K, T, D>*, node<K, T, D>*>(cur, tmp.second)
306.     };
307.     else {
308.         std::pair<node<K, T, D>*, node<K, T, D>*> tmp = split(cur-
>getleft(), x);
309.         cur->setleft(tmp.second);
310.         if (tmp.second != NULL)
311.             tmp.second->setparent(cur);
312.         if (tmp.first != NULL)
313.             tmp.first->setparent(NULL);
314.         return std::pair<node<K, T, D>*, node<K, T, D>*>(tmp.first, cur);
315.     }
316. }
317.
318. void cutnode(node<K, T, D>* n) {
319.     n->setparent(NULL);
320.     n->setleft(NULL);
321.     n->setright(NULL);
322. }
323.
324. void free(node<K, T, D>* t) {
325.     if (t == NULL) return;
326.     free(t->getleft());
327.     free(t->getright());
328.     delete t;
329. }
330.
331. public:
332.     treap<K, T, D>() {
333.         root = NULL;
334.     }
335.
336.     treap<K, T, D>(node<K, T, D>* newroot) {
337.         root = newroot;
338.     }
339.
340.     ~treap<K, T, D>() {
341.         free(root);
342.     }
343.     node<K, T, D>* merge(node<K, T, D>* R) {
344.         root = merge(root, R);
345.         return root;
346.     }
347.
348.     node<K, T, D>* merge(treap<K, T, D>& R) {
349.         root = merge(root, R.root);
350.         R.root = NULL;
351.         return root;
352.     }
353.
354.     //1)
355.     node<K, T, D>* merge(K key, T value, D data) {

```

```

356.     node<K, T, D>* tmp = new node<K, T, D>(key, value, data);
357.     return root = merge(root, tmp);
358. }
359.
360.     //добавление элемента
361.     node<K, T, D>* add(K key, T value, D data) {
362.         treap<K, T, D> L;
363.         treap<K, T, D> R;
364.         itersplit(L, R, key);
365.         L.merge(key, value, data);
366.         L.merge(R);
367.         std::swap(root, L.root);
368.         return root;
369.     }
370.
371.     //удаление элемента i hope... YES IT'S WORKING!
372.     node<K, T, D>* deleteElement(K key) {
373.         node<K, T, D> *less, *equal, *greater;
374.         less = split(getroot(), key).first;
375.         greater = split(getroot(), key).second;
376.
377.         equal = split(greater, key + 1).first;
378.         greater = split(greater, key + 1).second;
379.
380.         root = merge(less, greater);
381.         return root;
382.     }
383.
384.     //обычное разделение
385.     void split(treap<K, T, D>& L, treap<K, T, D>& R, K x) {
386.         if (L.root != NULL || R.root != NULL) throw condition_exeption();
387.         std::pair<node<K, T, D>*, node<K, T, D>*> tmp = split(root, x);
388.         L.root = tmp.first;
389.         R.root = tmp.second;
390.         root = NULL;
391.     }
392.
393.     //разделение через итераторы
394.     void itersplit(treap<K, T, D>& L, treap<K, T, D>& R, K x) {
395.         if (L.root != NULL || R.root != NULL) throw condition_exeption();
396.         for (treapiterator<K, T, D> iter = min(); iter != NULL; iter++) {
397.             if ((*iter)->getkey() < x) {
398.                 L.merge((*iter)->getkey(), (*iter)->getvalue(), (*iter)-
>getdata());
399.             }
400.             else {
401.                 R.merge((*iter)->getkey(), (*iter)->getvalue(), (*iter)-
>getdata());
402.             }
403.         }
404.     }
405.
406.     node<K, T, D>* min() {

```



```

407.     if (root == NULL) return NULL;
408.     node<K, T, D>* tmp = root;
409.     while (tmp->getleft() != NULL) {
410.         tmp = tmp->getleft();
411.     }
412.     return tmp;
413. }
414.
415. node<K, T, D>* max() {
416.     if (root == NULL) return NULL;
417.     node<K, T, D>* tmp = root;
418.     while (tmp->getright() != NULL) {
419.         tmp = tmp->getright();
420.     }
421.     return tmp;
422. }
423.
424. node<K, T, D>* getroot() {
425.     return root;
426. }
427.
428. friend std::ostream& operator<<(std::ostream& stream, treap<K, T, D
    >& treap) {
429.     for (treapiterator<K, T, D> iter = treap.min(); iter != NULL; iter
        ++) {
430.         stream << **iter;
431.     }
432.     return stream;
433. }
434.
435. friend std::ostream& operator<<(std::ostream& stream, const treap<K
    , T, D>& treap) {
436.     for (treapiterator<K, T, D> iter = treap.min(); iter != NULL; iter
        ++) {
437.         stream << **iter;
438.     }
439.     return stream;
440. }
441. };
442.
443. template<class T, class D>
444. class imptreap : protected treap<int, T, D> {
445. protected:
446.
447.     node<int, T, D>* merge(node<int, T, D>* L, node<int, T, D>* R) {
448.         if (L == NULL || R == NULL) return (L) ? L : R;
449.
450.         if (L->getvalue() > R->getvalue()) {
451.
452.             node<int, T, D>* tmp = merge(L->getright(), R);
453.             if (tmp != NULL)
454.                 tmp->setparent(L);
455.             L->setright(tmp);

```

```

456.     update(L);
457.     return L;
458. }
459. else {
460.     node<int, T, D>* tmp = merge(L, R->getleft());
461.     if (tmp != NULL)
462.         tmp->setparent(R);
463.     R->setleft(tmp);
464.     update(R);
465.     return R;
466. }
467. }
468.
469.     //обход дерева и переустановка ключей узлов на основе их порядковог
о номера в обходе
470.     int fix_keys(node<int, T, D>* ptr, int x) {
471.         if (ptr == NULL) return x;
472.         int tmp = fix_keys(ptr->getleft(), x);
473.         ptr->setkey(tmp);
474.         return fix_keys(ptr->getright(), tmp + 1);
475.     }
476.
477.     //обновление узлов в дереве, с помощью вычисления его размеров и п
роизведений значений узлов в его левом и правом поддеревьях
478.     void update(node<int, T, D>* n) {
479.         if (n == NULL) return;
480.         n->setsize(1 + ((n->getleft()) ? n->getleft()-
>getsize() : 0) + ((n->getright()) ? n->getright()->getsize() : 0));
481.         n->setproduct(n->getdata() * ((n->getleft()) ? n->getleft()-
>getproduct() : 1) * ((n->getright()) ? n->getright()->getproduct() : 1));
482.     }
483.
484.     std::pair<node<int, T, D>*, node<int, T, D>*> split(node<int, T, D>
* cur, int x) {
485.         if (cur == NULL) {
486.             return std::pair<node<int, T, D>*, node<int, T, D>*>(NULL, NULL);
487.         }
488.         int l = (cur->getleft()) ? cur->getleft()->getsize() : 0;
489.         if (l >= x) {
490.             std::pair<node<int, T, D>*, node<int, T, D>*> tmp = split(cur-
>getleft(), x);
491.             cur->setleft(tmp.second);
492.             if (tmp.second != NULL)
493.                 tmp.second->setparent(cur);
494.             if (tmp.first != NULL)
495.                 tmp.first->setparent(NULL);
496.             update(cur);
497.             return std::pair<node<int, T, D>*, node<int, T, D>*>(tmp.first, c
ur);
498.         }
499.         else {
500.             std::pair<node<int, T, D>*, node<int, T, D>*> tmp = split(cur-
>getright(), x - l - 1);

```

```

501.         cur->setright(tmp.first);
502.         if (tmp.second != NULL)
503.             tmp.second->setparent(NULL);
504.         if (tmp.first != NULL)
505.             tmp.first->setparent(cur);
506.         update(cur);
507.         return std::pair<node<int, T, D>*, node<int, T, D>*>(cur, tmp.second);
508.     }
509. }
510. public:
511.
512.     imptreap<T, D>() :treap<int, T, D>() {}
513.     imptreap<T, D>(node<int, T, D>* ptr) : treap<int, T, D>(ptr) {}
514.     ~imptreap<T, D>() {}
515.     node<int, T, D>* merge(imptreap<T, D>& R) {
516.         treap<int, T, D>::root = merge(treap<int, T, D>::root, R.root);
517.         R.root = NULL;
518.         return treap<int, T, D>::root;
519.     }
520.     node<int, T, D>* merge(T value, D data) {
521.         node<int, T, D>* tmp = new node<int, T, D>(1, value, data);
522.         return treap<int, T, D>::root = merge(treap<int, T, D>::root, tmp);
523.     }
524.
525.     void split(imptreap<T, D>& L, imptreap<T, D>& R, int x) {
526.         if (L.root != NULL || R.root != NULL) throw condition_exeption();
527.         std::pair<node<int, T, D>*, node<int, T, D>*> tmp = split(treap<int, T, D>::root, x - 1);
528.         L.root = tmp.first;
529.         R.root = tmp.second;
530.         treap<int, T, D>::root = NULL;
531.     }
532.
533.     void itersplit(imptreap<T, D>& L, imptreap<T, D>& R, int x) {
534.         if (L.root != NULL || R.root != NULL) throw condition_exeption();
535.         for (treapiterator<int, T, D> iter = min(); iter != NULL; iter++)
536.         {
537.             if ((*iter)->getkey() < x) {
538.                 L.merge((*iter)->getvalue(), (*iter)->getdata());
539.             }
540.             else {
541.                 R.merge((*iter)->getvalue(), (*iter)->getdata());
542.             }
543.         }
544.
545.         //произведение
546.         D find_product(int start, int end) {
547.             if (start > end || end < 0 || start < 0 || end > treap<int, T, D>::root->getsize())
548.                 || start > treap<int, T, D>::root->getsize())

```

```

549.         throw condition_exeption();
550.
551.         std::pair<node<int, T, D>*, node<int, T, D>*> tmp1 = split(treap<int, T, D>::root, start - 1);
552.         std::pair<node<int, T, D>*, node<int, T, D>*> tmp2 = split(tmp1.second, end - start + 1);
553.         D tmp = tmp2.first->getproduct();
554.         merge(tmp2.first, tmp2.second);
555.         merge(tmp1.first, tmp1.second);
556.         return tmp;
557.     }
558.
559.     //групповая функция
560.     int Maximum(int l, int r)
561.     {
562.         std::pair<node<int, T, D>*, node<int, T, D>*> p1;
563.         std::pair<node<int, T, D>*, node<int, T, D>*> p2;
564.         node<int, T, D>* QWERTY = getroot();
565.         p1 = split(QWERTY, r);
566.         p2 = split(p1.first, l);
567.         int res = p2.second->getproduct();
568.         QWERTY = merge(p2.first, merge(p2.second, p1.second));
569.         return res;
570.     }
571.
572.     int size(node<int, T, D>* ptr) {
573.         return ptr->getsize();
574.     }
575.
576.     node<int, T, D>* getroot() {
577.         return treap<int, T, D>::root;
578.     }
579.
580.     node<int, T, D>* min() {
581.         if (treap<int, T, D>::root == NULL) return NULL;
582.         node<int, T, D>* tmp = treap<int, T, D>::root;
583.         while (tmp->getleft() != NULL) {
584.             tmp = tmp->getleft();
585.         }
586.         return tmp;
587.     }
588.     node<int, T, D>* max() {
589.         if (treap<int, T, D>::root == NULL) return NULL;
590.         node<int, T, D>* tmp = treap<int, T, D>::root;
591.         while (tmp->getright() != NULL) {
592.             tmp = tmp->getright();
593.         }
594.         return tmp;
595.     }
596.     friend std::ostream& operator<<(std::ostream& stream, imptreap<T, D>& treap) {
597.         treap.fix_keys(treap.root, 1);

```

```

598.         for (treapiterator<int, T, D> iter = treap.min(); iter != NULL; it
           er++) {
599.             stream << **iter;
600.         }
601.         return stream;
602.     }
603.     friend std::ostream& operator<<(std::ostream& stream, const imptrea
        p<T, D>& treap) {
604.         treap.fix_keys(treap.root, 1);
605.         for (treapiterator<int, T, D> iter = treap.min(); iter != NULL; it
           er++) {
606.             stream << **iter;
607.         }
608.         return stream;
609.     }
610. };
611.
612.
613.     int main() {
614.         treap<int, int, int> treap1;
615.         treap1.merge(1, 1, 1);
616.         treap1.merge(2, 5, 1);
617.         treap1.merge(3, 2, 1);
618.         treap1.merge(4, 7, 1);
619.         treap1.merge(5, 3, 1);
620.         treap1.merge(6, 6, 1);
621.         treap1.merge(7, 4, 1);
622.
623.         treap<int, int, int> treap2;
624.         treap2.merge(1, 1, 1);
625.         treap2.merge(2, 5, 1);
626.         treap2.merge(3, 2, 1);
627.         treap2.merge(4, 7, 1);
628.         treap2.merge(5, 3, 1);
629.         treap2.merge(6, 6, 1);
630.         treap2.merge(7, 4, 1);
631.
632.         std::cout << "Treap\n" << treap2 << " root : " << *treap2.getroot()
           ;
633.         treap2.add(8, 9, 1);
634.         std::cout << "\nTreap after adding an element\n" << treap2 << " roo
           t : " << *treap2.getroot();
635.         treap2.deleteElement(6);
636.         std::cout << "\nTreap after deleting an element\n" << treap2 << " r
           oot : " << *treap2.getroot();
637.
638.         std::cout << "\n";
639.
640.         //std::cout << treap1 << " " << *treap1.getroot();
641.         std::cout << treap2 << " " << *treap2.getroot();
642.         std::cout << std::endl << " |-----
           -----| " << std::endl;
643.

```

```

644.      //treap1.merge(8, 9, 1);
645.
646.      //treap<int, int, int> treap3 = merge2(*treap1.getroot(), *treap2.g
        etroot());
647.      //treap1.merge2(treap1.getroot(), );
648.      //treap1.deleteElement(treap1.getroot(), 5);
649.
650.      std::cout << treap1 << " " << *treap1.getroot();
651.
652.      std::cout << std::endl << "|-----
        -----|" << std::endl;
653.
654.      std::cout << std::endl << "-----
        -----" << std::endl;
655.
656.      treap<int, int, int> left1;
657.      treap<int, int, int> right1;
658.      treap1.split(left1, right1, 5);
659.      std::cout << left1 << std::endl << right1 << std::endl;
660.
661.      std::cout << std::endl << "-----
        -----" << std::endl;
662.      right1.merge(left1);
663.      std::cout << right1;
664.
665.      std::cout << std::endl << "=====
        =====" << std::endl;
666.
667.      treap<int, int, int> left2;
668.      treap<int, int, int> right2;
669.      treap2.itorsplit(left2, right2, 5);
670.      std::cout << left2 << std::endl << right2 << std::endl;
671.
672.      std::cout << std::endl << "-----
        -----" << std::endl;
673.      right2.merge(left2);
674.      std::cout << right2;
675.
676.      std::cout << std::endl << "|||||
        -----" << std::endl;
677.      imptreap<int, double> implicatedtreap;
678.      implicatedtreap.merge(1, 1.5);
679.      implicatedtreap.merge(2, 5.5);
680.      implicatedtreap.merge(3, 2.5);
681.      implicatedtreap.merge(4, 7.0);
682.      implicatedtreap.merge(5, 3.0);
683.      implicatedtreap.merge(6, 6.001);
684.      implicatedtreap.merge(7, 4.1415);
685.
686.      std::cout << "ImpTreap:\n" << implicatedtreap << std::endl;
687.
688.      imptreap<int, double> left, right;
689.      implicatedtreap.itorsplit(left, right, 5);

```

```

690.     std::cout << "left:\n" << left << std::endl << "right:\n" << right
      << std::endl << "original:\n" << implicatedtreap << std::endl;
691.
692.     std::cout << "size of right root: " << right.size(right.getroot())
      << std::endl;
693.     left.merge(right);
694.     std::cout << "new left:\n" << left << std::endl;
695.     std::cout << std::endl << "-----
      -----" << std::endl;
696.     int a = 1, b = 7;
697.     std::cout << "product of left on [ " << a << " , " << b << " ]: " <
      < left.find_product(a, b) << std::endl;
698.
699.     std::cout << std::endl << "-----
      -----" << std::endl;
700.     treap<int, int, int> tr;
701.     for (int i = 0; i < 15; i++) {
702.         tr.add(i, rand() % 100, i);
703.     }
704.     std::cout << tr;
705.
706.     return 0;
707. }

```